

EC2x&EG2x-G&EG9x 系列

QuecOpen SMS API 参考手册

LTE Standard 模块系列

版本：1.0

日期：2023-05-17

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2023-02-21	Colin CUI	文档创建
1.0	2023-05-17	Colin CUI	受控版本

目录

文档历史	3
目录	4
表格索引	5
1 引言	6
1.1. 适用模块	6
2 SMS API.....	7
2.1. 头文件	7
2.2. 函数概览	7
2.3. 函数详解	8
2.3.1. QL_SMS_Client_Init.....	8
2.3.2. QL_SMS_Client_Deinit	8
2.3.3. QL_SMS_GetSmsCenterAddress	9
2.3.3.1. ql_sms_service_center_cfg_t	9
2.3.4. QL_SMS_SetSmsCenterAddress.....	10
2.3.5. QL_SMS_Send_Sms.....	10
2.3.5.1. ql_sms_info_t	11
2.3.5.2. ql_sms_user_data_head_t.....	12
2.3.6. QL_SMS_AddRxMsgHandler	13
2.3.6.1. QL_SMS_RxMsgHandlerFunc_t.....	13
2.3.7. QL_SMS_Send_SmsPdu	14
2.3.7.1. ql_wms_send_raw_message_data_t.....	14
2.3.7.2. ql_wms_raw_send_resp_t	15
2.3.7.3. E_QL_WMS_TL_CAUSE_CODE_TYPE	15
3 示例	19
3.1. 获取短信中心号码及类型	19
3.2. 发送文本短信	20
3.3. 接收文本短信	22
3.4. 发送 PDU 短信	25
4 附录 参考文档及术语缩写	27

表格索引

表 1: 适用模块	6
表 2: 函数概览	7
表 3: 参考文档	27
表 4: 术语缩写	27

1 引言

移远通信 LTE Standard EC2x 系列、EG2x-G 和 EG9x 系列模块支持 QuecOpen®方案；QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

短信是用户通过手机或其他通信终端直接发送或接收的文字或数字信息。移远通信 EC2x 系列、EG2x-G 和 EG9x 系列模块 QuecOpen®方案支持文本和 PDU 模式的 SMS。

本文档主要介绍如何通过移远通信 EC2x 系列、EG2x-G 和 EG9x 系列模块 QuecOpen® SDK 中提供的 SMS API 实现以下功能：

1. 获取短信中心号码及类型；
2. 设置短信中心号码及类型；
3. 发送文本短信（支持长短信发送）；
4. 接收文本短信（支持长短信接收）；
5. 发送 PDU 短信。

1.1. 适用模块

表 1：适用模块

模块系列	模块
EC2x	EC20-CE
	EC20-CN
	EC21 系列
	EC25 系列
EG2x-G	EG21-G
	EG25-G
EG9x	EG91 系列
	EG95 系列

2 SMS API

2.1. 头文件

SMS API 头文件为 `ql_mcm_sms.h`，位于 SDK 包的 `ql-ol-sdk/ql-ol-extsdk/include` 目录下。若无特别说明，本文档所涉及头文件均在该目录下。

2.2. 函数概览

表 2：函数概览

函数	说明
<code>QL_SMS_Client_Init()</code>	初始化 SMS 获取句柄
<code>QL_SMS_Client_Deinit()</code>	注销 SMS
<code>QL_SMS_GetSmsCenterAddress()</code>	获取短信中心号码及类型
<code>QL_SMS_SetSmsCenterAddress()</code>	设置短信中心号码及类型
<code>QL_SMS_Send_Sms()</code>	发送文本短信
<code>QL_SMS_AddRxMsgHandler()</code>	设置文本短信接收回调函数
<code>QL_SMS_Send_SmsPdu()</code>	发送 PDU 短信

备注

若无特别说明，所有 SMS 接口函数均不支持并发调用，并且不能在相关回调函数中调用以上 API。

2.3. 函数详解

2.3.1. QL_SMS_Client_Init

该函数用于初始化 SMS 获取句柄。

- 函数原型

```
int QL_SMS_Client_Init(sms_client_handle_type *ph_sms)
```

- 参数

ph_sms:
[Out] SMS 句柄指针。

- 返回值

<i>E_QL_SUCCESS</i>	函数执行成功
其他值	函数执行失败；错误码详见 <i>ql_mcm.h</i>

备注

使用其他 SMS 接口函数前，必须调用该函数初始化 SMS 获取句柄。

2.3.2. QL_SMS_Client_Deinit

该函数用于注销 SMS。

- 函数原型

```
int QL_SMS_Client_Deinit(sms_client_handle_type h_sms)
```

- 参数

h_sms:
[In] *QL_SMS_Client_Init()* 返回的 SMS 句柄。

- 返回值

<i>E_QL_SUCCESS</i>	函数执行成功
其他值	函数执行失败；错误码详见 <i>ql_mcm.h</i>

备注

若不再使用 SMS，必须调用该函数注销 SMS 以释放资源。

2.3.3. QL_SMS_GetSmsCenterAddress

该函数用于获取短信中心号码及类型。

● 函数原型

```
int QL_SMS_GetSmsCenterAddress( sms_client_handle_type h_sms, ql_sms_service_center_c
fg_t *get_sca_cfg)
```

● 参数

h_sms:

[In] QL_SMS_Client_Init()返回的 SMS 句柄。

get_sca_cfg:

[Out] 短信中心号码及类型；详见第2.3.3.1 章。

● 返回值

E_QL_SUCCESS 函数执行成功

其他值 函数执行失败；错误码详见 *ql_mcm.h*

2.3.3.1. ql_sms_service_center_cfg_t

短信中心号码及类型结构体定义如下：

```
typedef struct
{
    char service_center_addr[QL_SMS_MAX_ADDR_LENGTH + 1];
    uint8_t service_center_addr_type_valid;
    char service_center_addr_type[QL_SMS_MAX_SCA_TYPE_LENGTH + 1];
} ql_sms_service_center_cfg_t
```

● 参数

类型	参数	描述
char	<i>service_center_addr</i>	短信中心号码

		<i>service_center_addr_type</i> 是否有效
uint8_t	<i>service_center_addr_type_valid</i>	0 无效
		1 有效
char	<i>service_center_addr_type</i>	如果短信中心号码以“+”开头，填入“145”；否则填入“129”

2.3.4. QL_SMS_SetSmsCenterAddress

该函数用于设置短信中心号码及类型。

- 函数原型

```
int QL_SMS_SetSmsCenterAddress( sms_client_handle_type h_sms, ql_sms_service_center_cfg_t *set_sca_cfg)
```

- 参数

h_sms:

[In] QL_SMS_Client_Init()返回的 SMS 句柄。

set_sca_cfg:

[In] 短信中心号码及类型；详见第2.3.3.1章。

- 返回值

E_QL_SUCCESS 函数执行成功

其他值 函数执行失败；错误码详见 *ql_mcm.h*

备注

不推荐使用该函数设置短信中心号码及类型；因为若设置错误，可能导致无法发送短信。

2.3.5. QL_SMS_Send_Sms

该函数用于发送文本短信。

- 函数原型

```
int QL_SMS_Send_Sms(sms_client_handle_type h_sms, ql_sms_info_t *pt_sms_info)
```

● 参数

h_sms:

[In] *QL_SMS_Client_Init()*返回的 SMS 句柄。

pt_sms_info:

[In] 发送的短信内容、目的号码等；详见第2.3.5.1 章。

● 返回值

E_QL_SUCCESS 函数执行成功

其他值 函数执行失败；错误码详见 *ql_mcm.h*

2.3.5.1. ql_sms_info_t

发送的短信内容、目的号码等结构体定义如下：

```
typedef struct
{
    E_QL_SMS_STORAGE_TYPE_T e_storage;
    E_QL_SMS_FORMAT_T format;
    E_QL_SMS_TYPE_T type;
    char src_addr[QL_SMS_MAX_ADDR_LENGTH];
    int sms_data_len;
    char sms_data[QL_SMS_MAX_MT_MSG_LENGTH];
    char timestamp[21];
    uint8_t user_data_head_valid;
    ql_sms_user_data_head_t user_data_head;
    E_QL_SMS_MODE_TYPE_T e_mode;
    uint32_t storage_index;
} ql_sms_info_t
```

● 参数

类型	参数	描述
E_QL_SMS_STORAGE_TYPE_T	<i>e_storage</i>	短信息存储类型 <i>E_QL_SMS_STORAGE_TYPE_NONE</i> : 不存储 <i>E_QL_SMS_STORAGE_TYPE_UIM</i> : 存储在 UIM <i>E_QL_SMS_STORAGE_TYPE_NV</i> : 存储在 NVM
E_QL_SMS_FORMAT_T	<i>format</i>	短信格式 <i>E_QL_SMS_FORMAT_GSM_7BIT</i> : GSM-7 bit <i>E_QL_SMS_FORMAT_BINARY_DATA</i> : 二进制短信 <i>E_QL_SMS_FORMAT_UCS2</i> : UCS-2 编码短信

		<i>E_QL_SMS_FORMAT_IRA</i> : 不支持
<i>E_QL_SMS_TYPE_T</i>	<i>type</i>	短信类型 <i>E_QL_SMS_TYPE_RX</i> : 接收到的短信 <i>E_QL_SMS_TYPE_TX</i> : 发送的短信 <i>E_QL_SMS_TYPE_BROADCAST_RX</i> : 接收到的广播短信
char	<i>src_addr</i>	短信中心号码
int	<i>sms_data_len</i>	短信长度
char	<i>sms_data</i>	短信内容
char	<i>timestamp</i>	短信时间戳 格式: yy/MM/dd,hh:mm:ss+/-TimeZone
uint8_t	<i>user_data_head_valid</i>	<i>user_data_head</i> 是否有效 <i>TRUE</i> 有效 <i>FALSE</i> 无效
<i>ql_sms_user_data_head_t</i>	<i>user_data_head</i>	长短信头部信息; 详见第2.3.5.2章
<i>E_QL_SMS_MESSAGE_MODE_T</i>	<i>e_mode</i>	短信模式 <i>E_QL_SMS_MESSAGE_MODE_UNKNOWN</i> : 未知 <i>E_QL_SMS_MESSAGE_MODE_CDMA</i> : CDMA <i>E_QL_SMS_MESSAGE_MODE_GW</i> : GW
uint32_t	<i>storage_index</i>	存储索引; -1 表示不存储

2.3.5.2. ql_sms_user_data_head_t

长短信头部信息结构体定义如下:

```
typedef struct
{
    uint8_t total_segments;
    uint8_t seg_number;
    uint8_t reference_number;
}ql_sms_user_data_head_t
```

● 参数

类型	参数	描述
uint8_t	<i>total_segments</i>	长短信的总段数

uint8_t	<i>seg_number</i>	本条短信的段数编号
uint8_t	<i>reference_number</i>	本条短信的接收编号，本条短信的唯一标识号

2.3.6. QL_SMS_AddRxMsgHandler

该函数用于设置文本短信接收回调函数。

- 函数原型

```
int QL_SMS_AddRxMsgHandler(QL_SMS_RxMsgHandlerFunc_t handlerPtr, void* contextPtr)
```

- 参数

handlerPtr:

[In] 文本短信接收回调函数；详见第2.3.6.1章。

contextPtr:

[In] 上下文信息。作为参数传入到回调函数。

- 返回值

E_QL_SUCCESS 函数执行成功

其他值 函数执行失败；错误码详见 *ql_mcm.h*

2.3.6.1. QL_SMS_RxMsgHandlerFunc_t

文本短信接收回调函数，定义如下：

```
typedef void (*QL_SMS_RxMsgHandlerFunc_t)
(
    QL_SMS_MsgRef      msgRef,
    void*              contextPtr
)
```

- 参数

msgRef:

[In] 文本短信；详见第2.3.5.1章。

contextPtr:

[In] 上下文信息。设置文本短信接收回调函数时由用户传入。

2.3.7. QL_SMS_Send_SmsPdu

该函数用于发送 PDU 短信。

● 函数原型

```
int    QL_SMS_Send_SmsPdu(    sms_client_handle_type    h_sms,
ql_wms_send_raw_message_data_t *raw_message_data, ql_wms_raw_send_resp_t *rawresp)
```

● 参数

h_sms:

[In] *QL_SMS_Client_Init()*返回的 SMS 句柄。

raw_message_data:

[In] PDU 短信；详见第2.3.7.1 章。

rawresp:

[In] PDU 短信发送结果；详见第2.3.7.2 章。

● 返回值

E_QL_SUCCESS 函数执行成功

其他值 函数执行失败；错误码详见 *ql_mcm.h*

2.3.7.1. ql_wms_send_raw_message_data_t

PDU 短信结构体定义如下：

```
typedef struct
{
    E_QL_WMS_MESSAGE_FORMAT_TYPE format;
    uint32_t raw_message_len;
    uint8_t raw_message[QL_WMS_MESSAGE_LENGTH_MAX];
}ql_wms_send_raw_message_data_t
```

● 参数

类型	参数	描述
E_QL_WMS_MESSAGE_FORMAT_TYPE	<i>format</i>	PDU 短信格式 E_QL_WMS_MESSAGE_FORMAT_CDMA: CDMA E_QL_WMS_MESSAGE_FORMAT_GW_PP:

GW – 点对点		
uint32_t	<i>raw_message_len</i>	PDU 短信长度
uint8_t	<i>raw_message</i>	PDU 短信内容

2.3.7.2. ql_wms_raw_send_resp_t

PDU 短信发送结果结构体定义如下：

```
typedef struct
{
    uint16_t    message_id;
    uint8_t     cause_code_valid;
    E_QL_WMS_TL_CAUSE_CODE_TYPE    cause_code;
}ql_wms_raw_send_resp_t
```

● 参数

类型	参数	描述
uint16_t	<i>message_id</i>	本条 PDU 短信的 ID
uint8_t	<i>cause_code_valid</i>	<i>cause_code</i> 是否有效 0 无效 1 有效
<i>E_QL_WMS_TL_CAUSE_CODE_TYPE</i>	<i>cause_code</i>	短信发送失败的错误码；详见第2.3.7.3章

2.3.7.3. E_QL_WMS_TL_CAUSE_CODE_TYPE

短信发送失败的错误码枚举定义如下：

```
typedef enum
{
    E_QL_WMS_TL_CAUSE_CODE_ADDR_VACANT = 0x00,
    E_QL_WMS_TL_CAUSE_CODE_ADDR_TRANSLATION_FAILURE = 0x01,
    E_QL_WMS_TL_CAUSE_CODE_NETWORK_RESOURCE_SHORTAGE = 0x02,
    E_QL_WMS_TL_CAUSE_CODE_NETWORK_FAILURE = 0x03,
    E_QL_WMS_TL_CAUSE_CODE_INVALID_TELESERVICE_ID = 0x04,
    E_QL_WMS_TL_CAUSE_CODE_NETWORK_OTHER = 0x05,
    E_QL_WMS_TL_CAUSE_CODE_NO_PAGE_RESPONSE = 0x20,
```

```

E_QL_WMS_TL_CAUSE_CODE_DEST_BUSY                = 0x21,
E_QL_WMS_TL_CAUSE_CODE_NO_ACK                    = 0x22,
E_QL_WMS_TL_CAUSE_CODE_DEST_RESOURCE_SHORTAGE    = 0x23,
E_QL_WMS_TL_CAUSE_CODE_SMS_DELIVERY_POSTPONED    = 0x24,
E_QL_WMS_TL_CAUSE_CODE_DEST_OUT_OF_SERV         = 0x25,
E_QL_WMS_TL_CAUSE_CODE_DEST_NOT_AT_ADDR         = 0x26,
E_QL_WMS_TL_CAUSE_CODE_DEST_OTHER               = 0x27,
E_QL_WMS_TL_CAUSE_CODE_RADIO_IF_RESOURCE_SHORTAGE = 0x40,
E_QL_WMS_TL_CAUSE_CODE_RADIO_IF_INCOMPATABILITY  = 0x41,
E_QL_WMS_TL_CAUSE_CODE_RADIO_IF_OTHER           = 0x42,
E_QL_WMS_TL_CAUSE_CODE_ENCODING                 = 0x60,
E_QL_WMS_TL_CAUSE_CODE_SMS_ORIG_DENIED          = 0x61,
E_QL_WMS_TL_CAUSE_CODE_SMS_TERM_DENIED          = 0x62,
E_QL_WMS_TL_CAUSE_CODE_SUPP_SERV_NOT_SUPP       = 0x63,
E_QL_WMS_TL_CAUSE_CODE_SMS_NOT_SUPP             = 0x64,
E_QL_WMS_TL_CAUSE_CODE_MISSING_EXPECTED_PARAM   = 0x65,
E_QL_WMS_TL_CAUSE_CODE_MISSING_MAND_PARAM       = 0x66,
E_QL_WMS_TL_CAUSE_CODE_UNRECOGNIZED_PARAM_VAL   = 0x67,
E_QL_WMS_TL_CAUSE_CODE_UNEXPECTED_PARAM_VAL     = 0x68,
E_QL_WMS_TL_CAUSE_CODE_USER_DATA_SIZE_ERR       = 0x69,
E_QL_WMS_TL_CAUSE_CODE_GENERAL_OTHER            = 0x6A,
}E_QL_WMS_TL_CAUSE_CODE_TYPE

```

● 成员

成员	描述
<i>E_QL_WMS_TL_CAUSE_CODE_ADDR_VACANT</i>	SMS 目的地址有效，但当前未分配给 SMS 终端；与有效目的地址关联的 MIN 对其 HLR 是未知的
<i>E_QL_WMS_TL_CAUSE_CODE_ADDR_TRANSLATION_FAILURE</i>	SMS 目的地址无效。例如： ● 该地址不是可识别的地址类型 ● 该地址不是针对已知或可能的 SMS 功能实体 ● 目的 MS 地址关联的 MIN 与其 HLR 不对应 ● 目的 MS 关联的 ESN 与期望值不匹配 ● SMS 目的地址（SMS_DestinationAddress）、SMS 起始地址（SMS_OriginalDestinationAddress）、目的 MIN 或原始目的子地址与目的 SME 的地址不匹配
<i>E_QL_WMS_TL_CAUSE_CODE_NETWORK_RESOURCE_SHORTAGE</i>	由于缺乏网络资源或链路容量，网络传输失败
<i>E_QL_WMS_TL_CAUSE_CODE_NETWORK_FAILURE</i>	网络节点失败、链接失败或所需操作失败
<i>E_QL_WMS_TL_CAUSE_CODE_INVALID_TELESERVICE_ID</i>	SMS_TeleserviceIdentifier 未知、不受支持或未被寻址的功能实体授权

E_QL_WMS_TL_CAUSE_CODE_NETWORK_OTHER	其他网络问题
E_QL_WMS_TL_CAUSE_CODE_NO_PAGE_RESPONSE	被寻址的 MS-based SME 是已知的,但未响应寻呼;SMS 通知未挂起
E_QL_WMS_TL_CAUSE_CODE_DEST_BUSY	目标 MS-based SME 具有 SMS 能力,但当前正在进行呼叫、服务或呼叫模式,无法使用 SMS,或者目的 SME 拥塞。该值只能在双边协议允许的情况下在 MSC 和 MC 之间使用。SMS 通知未挂起
E_QL_WMS_TL_CAUSE_CODE_NO_ACK	目的 SME 未确认收到 SMS 传送
E_QL_WMS_TL_CAUSE_CODE_DEST_RESOURCE_SHORTAGE	所需的终端资源(内存等)不可用于处理此消息;SMS 通知未挂起
E_QL_WMS_TL_CAUSE_CODE_SMS_DELIVERY_POSTPONED	当前无法发送(例如无页面响应、目的地忙、无确认、目的地停止服务、其他终端问题),但 SMS 通知待处理
E_QL_WMS_TL_CAUSE_CODE_DEST_OUT_OF_SERV	被寻址的目标长时间无服务(例如 MS 睡眠、不活动、关机);SMS 通知未挂起
E_QL_WMS_TL_CAUSE_CODE_DEST_NOT_AT_ADDR	MS-based SME 不再位于临时 SMS 路由地址。消息发送者不应重复使用临时 SMS 路由地址。SMS 通知未挂起
E_QL_WMS_TL_CAUSE_CODE_DEST_OTHER	其他终端问题;SMS 通知未挂起
E_QL_WMS_TL_CAUSE_CODE_RADIO_RESOURCE_SHORTAGE	无可可用通道或无线电拥塞
E_QL_WMS_TL_CAUSE_CODE_RADIO_INCOMPATABILITY	MS-based SME 的 MS 在不支持 SMS 的模式下运行
E_QL_WMS_TL_CAUSE_CODE_RADIO_OTHER	其他 MS-based SME 的无线电接口问题
E_QL_WMS_TL_CAUSE_CODE_ENCODING	非预期的参数或字段大小
E_QL_WMS_TL_CAUSE_CODE_SMS_ORIG_DENIED	原始 MIN 不被识别、原始 MIN 不允许使用原始地址、ESN 与原始 MIN 不匹配、原始未被授权、原始地址不被识别等
E_QL_WMS_TL_CAUSE_CODE_SMS_TERM_DENIED	目标无权接收 SMS 消息、MC 拒绝消息、目的 SME 拒绝消息、目标无权获得所需的补充服务等
E_QL_WMS_TL_CAUSE_CODE_SUPP_SERV_NOT_SUPP	原始补充服务未知或不支持,发件人无权使用原始补充服务等
E_QL_WMS_TL_CAUSE_CODE_SMS_NOT_SUPP	寻址的功能实体不支持 SMS
E_QL_WMS_TL_CAUSE_CODE_MISSING_EXPECTED_PARAM	特定功能所需的可选参数
E_QL_WMS_TL_CAUSE_CODE_MISSING_MAND_PARAM	缺少特定消息必需的参数
E_QL_WMS_TL_CAUSE_CODE_UNRE	已知参数具有未知或不受支持的值

COGNIZED_PARAM_VAL

E_QL_WMS_TL_CAUSE_CODE_UNEX
PECTED_PARAM_VAL

已知参数具有已知但非预期的值

E_QL_WMS_TL_CAUSE_CODE_USER
_DATA_SIZE_ERR

用户数据量对于接入技术、传输网络或呼叫方式等来说过大；用户数据大小非指定远程服务的预期大小

E_QL_WMS_TL_CAUSE_CODE_GENE
RAL_OTHER

其他问题

3 示例

本章所有示例均位于 `ql-ol-sdk/ql-ol-extsdk/example/test_mcm_api/test_sms.c` 或 `ql-ol-sdk/ql-ol-extsdk/example/sms/example_sms.c` 中。用户可以自行在列出的目录中查看 SMS API 的完整示例。

1. 程序启动后，必须调用 `QL_SMS_Client_Init()` 初始化 SMS 获取句柄。
2. 程序退出前或不使用 SMS 时，必须调用 `QL_SMS_Client_Deinit()` 注销 SMS 以释放资源。

3.1. 获取短信中心号码及类型

示例目录：`ql-ol-sdk/ql-ol-extsdk/example/sms/example_sms.c`。

```
case 63:
{
    ql_sms_service_center_cfg_t get_sca_cfg;
    memset(&get_sca_cfg,0,sizeof(get_sca_cfg));

    iResult = QL_SMS_GetSmsCenterAddress(h_exsms,&get_sca_cfg);

    if(iResult < 0)
    {
        printf("QL_SMS_GetSmsCenterAddress. iResult:%d\n",iResult);
    }
    else
    {
        printf("ervice_center_addr:%s    service_center_addr_type:%s\n",
get_sca_cfg.service_center_addr,get_sca_cfg.service_center_addr_type);
    }
    break;
}
```

3.2. 发送文本短信

示例目录：`ql-ol-sdk/ql-ol-extsdk/example/test_mcm_api/test_sms.c`。

如下代码展示了如何发送 GSM-7 bit、二进制和 UCS-2 格式的文本短信。

```
case 1:/"QL_SMS_Send_Sms"
{
    int            i            = 0;
    int            len          = 0;
    E_QL_SMS_FORMAT_T    e_format= 0;
    char            sms_buf[QL_SMS_MAX_MT_MSG_LENGTH] = {0};
    ql_sms_info_t    *pt_sms_info = NULL;

    pt_sms_info = (ql_sms_info_t*)malloc(sizeof(ql_sms_info_t));
    if(pt_sms_info == NULL)
    {
        printf("Malloc fail!\n");
        break;
    }
    memset(pt_sms_info, 0, sizeof(ql_sms_info_t));

    printf("please input dest phone number: \n");
    scanf("%s", pt_sms_info->src_addr);

    printf("please input sms encoding type(0:GSM-7, 1:Binary, 2:UCS2, 3:IRA): \n");
    scanf("%d", &e_format);
    e_format = e_format & 0x03;

    quectel_set_recvsms_format(e_format); //config recvice sms format

    if((e_format == E_QL_SMS_FORMAT_GSM_7BIT) || (e_format ==
E_QL_SMS_FORMAT_IRA) || (e_format == E_QL_SMS_FORMAT_UCS2))
    {
        printf("please input message content: \n");
        getchar();//Add this on purpose or the following fgets will be skipped.
        fgets(sms_buf, QL_SMS_MAX_MT_MSG_LENGTH, stdin);
        len= strlen(sms_buf); //to skip 0x0A
        sms_buf[len-1] = '\0';
        printf("textData[%d]:%s\n", len, sms_buf);

        printf("input %d byte data:", len);
        for(i=0; i<len; i++)
```

```

        {
            printf("%.2X ", sms_buf[i]);
        }
        printf("\n");
    }
    else
    {
        printf("please input binary data counts in bytes: \n");
        do
        {
            i = scanf("%d", &len);
        }while(i != 1);

        printf("please input binary data: \n");
        for(i=0; i<len; i++)
        {
            printf("Byte[%d]=", i);
            scanf("%2X", &sms_buf[i]);
        }
    }

    if(e_format == E_QL_SMS_FORMAT_GSM_7BIT || e_format ==
E_QL_SMS_FORMAT_IRA)
    {
        memcpy(pt_sms_info->sms_data, sms_buf,
QL_SMS_MAX_MT_MSG_LENGTH);
        pt_sms_info->sms_data_len = strlen(sms_buf);
    }
    else if(e_format == E_QL_SMS_FORMAT_BINARY_DATA)
    {
        memcpy(pt_sms_info->sms_data, sms_buf, len);
        pt_sms_info->sms_data_len = len;
    }
    else
    {
        len = UTF8StrToUnicodeStr(sms_buf,
                                (uint16_t*)pt_sms_info->sms_data,
                                len); // Got BE data
        pt_sms_info->sms_data_len = len * 2; // in bytes

        printf("raw UTF-16 len=%d, data:", pt_sms_info->sms_data_len);
        for(i=0; i<pt_sms_info->sms_data_len; i++)
        {
            printf("%.2X ", pt_sms_info->sms_data[i]);
        }
    }
}

```

```

    }
    printf("\n");
}

pt_sms_info->format = e_format;

ret = QL_SMS_Send_Sms(h_sms, pt_sms_info);
printf("#QL_SMS_Send_Sms ret=%d \n", ret);
free(pt_sms_info);
break;
}          break;
}

```

3.3. 接收文本短信

示例目录：*ql-ol-sdk/ql-ol-extsdk/example/test_mcm_api/test_sms.c*。

文本短信接收回调函数代码如下：

```

static void ql_sms_cb_func( QL_SMS_MsgRef  msgRef,
                           void*          contextPtr)
{
    int i;
    if(msgRef->e_storage != E_QL_SMS_STORAGE_TYPE_NONE)
    {
        char *msg_format[] = {"CDMA", "GW"};
        char *storage_type[] = {"UIM", "NV"};
        printf("###You've got one new %s message, stored to %s index=%d\n",
               msg_format[msgRef->e_mode & 1],
               storage_type[msgRef->e_storage & 1],
               msgRef->storage_index);
    }
    else if(msgRef->format == E_QL_SMS_FORMAT_UCS2)
    {
        unsigned char* smsbuf = NULL;

        smsbuf = (char*)malloc(sizeof(char)*QL_SMS_MAX_MT_MSG_LENGTH);
        if(smsbuf == NULL)
        {
            printf("Out of memory");
            return;
        }
    }
}

```

```

    }
    memset(smsbuf, 0, QL_SMS_MAX_MT_MSG_LENGTH);
    UnicodeStrToUTF8Str((unsigned short*)&msgRef->sms_data[0],
                        smsbuf,
                        QL_SMS_MAX_MT_MSG_LENGTH);
    if (msgRef->user_data_head_valid)
    {
        printf("\n###You've got one new UCS2 msg from %s at %s,
total_segments:%d, seg_number:%d, reference_number:%02x, len=%d, content=%s\n",
                msgRef->src_addr,
                msgRef->timestamp,

msgRef->user_data_head.total_segments,

msgRef->user_data_head.seg_number,

msgRef->user_data_head.reference_number,

                msgRef->sms_data_len,
                smsbuf);
    }
    else
    {
        printf("\n###You've got one new UCS2 msg from %s at %s, len=%d,
content=%s\n",

                msgRef->src_addr,
                msgRef->timestamp,
                msgRef->sms_data_len,
                smsbuf);
    }
    printf("Received UCS raw data:");
    for(i=0; i<msgRef->sms_data_len; i++)
    {
        printf("%.2X ", msgRef->sms_data[i]);
    }

    printf("\nAfter convert to UTF8, len=%d, data:", strlen(smsbuf));
    for(i=0; i<strlen(smsbuf); i++)
    {
        printf("%.2X ", smsbuf[i]);
    }
    printf("\n");
    free(smsbuf);
}
else if(msgRef->format == E_QL_SMS_FORMAT_BINARY_DATA)

```

```

{
    if (msgRef->user_data_head_valid)
    {
        printf("###You've got one new BINARY msg from %s at %s , total_segments:%d,
seg_number:%d, reference_number:%02x, len=%d, content=",
                                msgRef->src_addr,
                                msgRef->timestamp,

msgRef->user_data_head.total_segments,

msgRef->user_data_head.seg_number,

msgRef->user_data_head.reference_number,

                                msgRef->sms_data_len);
    }
    else
    {
        printf("###You've got one new BINARY msg from %s at %s , len=%d, content=",
                                msgRef->src_addr,
                                msgRef->timestamp,
                                msgRef->sms_data_len);

    }
    for(i=0; i<msgRef->sms_data_len; i++)
    {
        printf("%.2X ", msgRef->sms_data[i]);
    }
    printf("\n");
}
else //default is GSM-7
{
    if (msgRef->user_data_head_valid)
    {
        printf("###You've got one new GSM-7 msg from %s at %s, total_segments:%d,
seg_number:%d, reference_number:%02x, content=%s\n",
                                msgRef->src_addr,
                                msgRef->timestamp,

msgRef->user_data_head.total_segments,
                                msgRef->user_data_head.seg_number,

msgRef->user_data_head.reference_number,
                                msgRef->sms_data);
    }
    else

```

```

        {
            printf("###You've got one new GSM-7 msg from %s at %s,
content=%s\n",
                    msgRef->src_addr,
                    msgRef->timestamp,
                    msgRef->sms_data);
        }
        for(i=0; i<msgRef->sms_data_len; i++)
        {
            printf("%.2X ", msgRef->sms_data[i]);
        }
        printf("\n\n");
    }
}

```

设置文本短信接收回调函数代码如下：

```

case 3:/"QL_SMS_AddRxMsgHandler"
{
    ret = QL_SMS_AddRxMsgHandler(ql_sms_cb_func, (void*)h_sms);
    printf("QL_SMS_AddRxMsgHandler ret=%d \n", ret);
    break;
}

```

3.4. 发送 PDU 短信

示例目录：*ql-ol-sdk/ql-ol-extsdk/example/sms/example_sms.c*。

```

case 62:
{
    char aPduMsg[255*2+1]={0};
    ql_wms_send_raw_message_data_t sWmsRawMsg;
    ql_wms_raw_send_resp_t sWmsRawResp;

    memset(&sWmsRawMsg,0,sizeof(sWmsRawMsg));
    memset(&sWmsRawResp,0,sizeof(sWmsRawResp));

    printf("please input PDU content: \n");
    scanf("%s",aPduMsg);
    if(false
long_hexstr_to_oct(aPduMsg,strlen(aPduMsg),sWmsRawMsg.raw_message,&sWmsRawMsg.ra
w_message_len,sizeof(sWmsRawMsg.raw_message)))

```

```

        {
            printf(" long_hexstr_to_oct FAIL\n");
            return -1;
        }

        print_hexstr(sWmsRawMsg.raw_message,sWmsRawMsg.raw_message_len);

        printf("please input PDU FORMAT(0-CAMD_PP; 6-GW_PP): \n");
        uint8_t uFormat = 0;
        scanf("%d",&uFormat);
        sWmsRawMsg.format = uFormat;

        printf("please FORMAT = %d \n",sWmsRawMsg.format);

        iResult =
QL_SMS_Send_SmsPdu(h_exsms,&sWmsRawMsg,&sWmsRawResp);
        if(iResult < 0)
        {
            printf("QL_SMS_Send_SmsPdu FAIL. iResult:%d\n",iResult);
        }
        else
        {
            printf(" message_id:%d\n",sWmsRawResp.message_id);
        }

        if(sWmsRawResp.cause_code_valid)
        {
            printf(" cause_code:%02x\n",sWmsRawResp.cause_code);
        }
    }
    break;

```

4 附录 参考文档及术语缩写

表 3：参考文档

文档名称
[1] Quectel_EC2x&EG2x-G&EG9x 系列_QuecOpen_快速开发指导

表 4：术语缩写

缩写	英文全称	中文全称
ASCII	American Standard Code for Information Interchange	美国信息交换标准代码
CDMA	Code Division Multiple Access	码分多址
ESN	Electronic Serial Number	电子序列号
GSM	Global System for Mobile Communications	全球移动通信系统
HLR	Home Location Register	归属位置寄存器
ISO	International Standard Organization	国际标准化组织
MC	Message Center	消息中心
MIN	Mobile Identification Number	移动识别号码
MS	Mobile Station	移动台站点
MSC	Mobile Switching Center	移动交换中心
NVM	Non-Volatile Memory	非易失性存储器
PDU	Protocol Data Unit	协议数据单元
SDK	Software Development Kit	软件开发工具包
SME	Station Management Entity	站点管理实体
SMS	Short Message Service	短信服务

UCS	Universal Character Set	通用字符集
UIM	User Identity Module	用户识别模块
WCDMA	Wideband CDMA	宽带码分多址